



GitHub Trending Top 10

每日热门开源项目深度解读

2026-09-26

今日榜单

- 1 paperclipai/paperclip** TypeScript 2天
The open-source app everyone uses to manage agents at work
- 2 vectorize-io/hindsight** Python 3天
Hindsight: Agent Memory That Learns
- 3 NVIDIA/Model-Optimizer** Python NEW
A unified library of SOTA model optimization techniques like quantization, disti...
- 4 dream-num/univer** TypeScript 5天
The Office Harness for AI Agents — Spreadsheets, Docs, Slides, Canvas, Relationa...
- 5 tensorflow/tensorflow** C++ NEW
An Open Source Machine Learning Framework for Everyone
- 6 rohitg00/ai-engineering-from-scratch** Python NEW
Learn it. Build it. Ship it for others.
- 7 openbao/openbao** Go NEW
OpenBao is a software solution to manage, store, and distribute sensitive data i...
- 8 block/buzz** Rust NEW
A hive mind communication platform
- 9 microsoft/vscode** TypeScript NEW
Visual Studio Code
- 10 zhaoxuya520/reverse-skill** PowerShell NEW
Reverse Engineering / Authorized Penetration Testing / Security Research Skill R...

#1

TypeScript

连续 2 天

paperclip

The open-source app everyone uses to manage agents at work

Stars **86,157** Forks **15,329** Today **+2,589**

<https://github.com/paperclipai/paperclip>

项目简介：Paperclip 是一个开源的 AI Agent 编排平台，定位为「管理 AI 员工的公司级应用」。它要解决的问题是：当团队同时运行多个 AI Agent（如 Claude Code、Codex、Cursor 等）时，如何像管理一家公司一样统一分配目标、协调工作、追踪成本和审核产出，而不是在几十个终端窗口间手忙脚乱。

核心功能：一是**目标驱动的任务管理**——你定义业务目标（如“做出百万 MRR 的笔记应用”），系统拆解并派发给不同 Agent；二是**组织架构与治理**——支持“雇佣”CEO、CTO、工程师等角色，设置预算和审批门禁；三是**成本监控与预算管控**——从统一仪表盘追踪各 Agent 的花费；四是**多 Agent 兼容**——通过心跳机制接入 OpenClaw、Claude、Codex、Cursor、Bash、HTTP 等任意 Agent；五是**审计与人工介入**——可查看 diff、随时插手干预，兼顾自主性与可控性。

适用场景：适合已经在用多个 AI 编程/自动化 Agent 的重度用户——尤其是那些“同时开着 20 个 Claude Code 终端、已经搞不清谁在干什么”的开发者。也适合想让 Agent 7×24 小时自主运转、但仍需审计成本和产出的团队，甚至支持从手机端管理“自主业务”。

技术亮点：架构上是一个 Node.js 服务端 + React UI，把「任务管理器」的表象和「组织架构、预算、治理、目标对齐、Agent 协调」的底层结合起来。其设计哲学颇具深意——“如果 OpenClaw 是员工，Paperclip 就是公司”，通过心跳（heartbeat）机制实现 Agent 的通用接入，抽象层次清晰，把 Agent 编排问题提升到了“组织管理”的高度。

上手建议：建议先阅读官方文档（docs.papercliping）和 README 中的 Quickstart，理解“定义目标 → 组建团队 → 审批运行”三步流程。可从接入单个 Agent（如 Claude Code 或 Bash）跑通最小闭环开始，再逐步扩展到多 Agent 协作。想贡献的话，可加入其 Discord 社区了解开发方向，项目为 MIT 协议、TypeScript 编写，前端 React 与后端 Node.js 都是常见技术栈，上手门槛较低。

#2

Python

连续 3 天

hindsight

Hindsight: Agent Memory That Learns

Stars **30,871** Forks **3,338** Today **+2,152**

<https://github.com/vectorize-io/hindsight>

项目简介：Hindsight 是由 Vectorize.io 推出的 Agent 记忆系统，目标是让 AI Agent 不仅能"记住"对话历史，更能从中"学习"并随时间进化。它针对 RAG 和知识图谱等传统方案在长期记忆任务上的不足，提供更准确、更智能的记忆能力，并在 LongMemEval 基准上取得了当前最优表现（SOTA）。

核心功能：

- **三类记忆操作：**通过 retain（保留）、recall（召回）、reflect（反思）三个原语管理 Agent 记忆，覆盖写入、检索与抽象学习全流程。
- **记忆类型与观察抽象：**支持多种记忆类型，并能从原始记忆中提炼出"observations"（观察结论），实现从"记住事实"到"形成认知"的跃迁。
- **Mental Models 与 Knowledge Pages：**构建结构化的心智模型和知识页面，让 Agent 拥有可复用的长期知识。
- **多端接入：**提供 LLM Wrapper（两行代码接入）、MCP Server、编码 Agent 集成，以及 Python 嵌入式（无需服务端）等多种使用方式。
- **Memory Banks：**以"记忆库"为单位隔离和管理不同上下文，适配多用户、多任务场景。

适用场景：

- 需要长期个性化记忆的对话式 AI（如客服、陪伴、教育助手）。
- 构建能跨会话积累经验的自主 Agent（如编码助手、研究助手）。
- 企业级生产环境（README 提到已在财富 500 强企业落地），需要高准确率长期记忆的 AI 应用。
- 使用 Claude Code、Cursor 等编码 Agent，希望通过 MCP 为其注入持久记忆的开发者。

技术亮点：

- 在 LongMemEval 上达到 SOTA，且结果被 Virginia Tech 和《华盛顿邮报》独立复现，可信度较高。
- 明确区分"记忆"与"学习"，通过 reflect/observations 机制实现记忆的抽象与沉淀，而非简单向量检索。
- 兼容 25+ LLM 提供商（含 OpenAI、Anthropic、Gemini 及本地 Ollama），部署灵活，支持 Docker 一键启动和 Python 内嵌两种模式。
- 同时提供 Python 与 NPM 客户端，工程化程度高，配套文档、Cookbook、Benchmark 站点齐全。

上手建议：

- 最快路径是用 Docker 启动服务端（设好 OPENAI_API_KEY 即可），然后通过 LLM Wrapper 两行代码接入现有 Agent。
- 想深入理解设计理念，建议先读 arXiv 论文（2512.12818）和官方文档中的 Core Concepts 部分。
- 贡献者可关注 GitHub 上的集成扩展（Integrations）和

Cookbook, 从补充示例或适配新 LLM 提供商入手门槛较低。 - 使用编码 Agent 的开发者可直接 `npx skills add` 安装文档技能, 边写代码边查文档。

#3

Python

NEW

Model-Optimizer

A unified library of SOTA model optimization techniques like quantization, distillation, pruning, neural architecture search, speculative decoding, etc. It compresses deep learning models for downstream deployment frameworks like TensorRT-LLM, TensorRT, vLLM, etc. to optimize inference speed.

Stars **4,622** Forks **659** Today **+354**

<https://github.com/NVIDIA/Model-Optimizer>

项目简介：NVIDIA Model Optimizer (ModelOpt) 是 NVIDIA 推出的统一模型优化库，把量化、蒸馏、剪枝、神经架构搜索、投机解码等多种压缩技术整合到一套 Python API 中。它解决的核心问题是：大模型推理又慢又占显存，需要在不明显损失精度的前提下压缩模型、提升推理速度，并让优化后的权重能直接落地到 TensorRT-LLM、vLLM、SGLang 等部署框架。

核心功能：

- **量化：**支持 FP8、NVFP4 (W4A4) 等低精度格式，提供 PTQ 与量化感知蒸馏 (QAD)，在压缩模型的同时尽量恢复精度。
- **剪枝与 NAS：**包括 Minitron 结构化剪枝、Puzzlettron 异构剪枝/NAS，可把大模型裁成更小的"子模型"。
- **蒸馏与投机解码：**支持两阶段蒸馏和 speculative decoding，进一步加速推理。
- **统一导出：**提供 Hugging Face 导出 API，同时支持 transformers 和 diffusers 模型，一键产出可部署的量化 checkpoint。

适用场景：主要面向需要把大模型 (LLM、VLM、扩散模型) 部署到生产环境的团队，尤其是在 NVIDIA GPU 上追求高吞吐、低显存占用的推理服务。典型用法是拿一个 Hugging Face / PyTorch / ONNX 模型，做量化或剪枝蒸馏后导出，再交给 TensorRT-LLM、vLLM 等框架上线。也适合做模型压缩研究的工程师，借助 Megatron-Bridge、Megatron-LM、Accelerate 等集成做训练阶段的优化。

技术亮点：

- **软硬件协同：**与 NVIDIA 生态深度绑定，NVFP4 等格式直接对齐最新 GPU 的低精度算力，README 中 Nemotron 3 Ultra 550B 量化后 decode 吞吐可达 BF16 的 5.9 倍。
- **精度恢复手段：**QAD (量化感知蒸馏)、Local-Hessian 权重 scale、AutoQuantize 自动混合精度分配，都是在激进量化下保住精度的关键。
- **端到端可复现：**官方提供了 Qwen3.6、Nemotron-3-Nano 等完整教程，从 PTQ 到部署链路都有示例。

上手建议：先读官方文档 (nvidia.github.io/Model-Optimizer) 和 Roadmap，从 examples/ 里的端到端教程入手，比如 Qwen3.6-35B 的 NVFP4 + QAD 或 Nemotron-3-Nano 的剪枝+蒸馏+FP8 流程，照着跑通一遍最直观。安装可直接 `pip install nvidia-modelopt`。想贡献的话，建议先看

GitHub Issues 里的 Roadmap (#1699) 和现有 examples 的写法，从补充教程、修 bug 或适配新模型/新框架这类切口进入。

#4

TypeScript

连续 5 天

univer

The Office Harness for AI Agents — Spreadsheets, Docs, Slides, Canvas, Relational Tables, and PDF in one runtime.

Stars **18,995** Forks **1,618** Today **+845**

<https://github.com/dream-num/univer>

项目简介：Univer 是一个开源的 Office SDK，让开发者能在自己的产品里嵌入电子表格、文档、演示文稿等办公能力，而不必依赖某个固定的托管应用或现成 UI。它解决的核心问题是：过去要自建类 Excel/Google Docs 体验，要么用重型商业组件，要么从零造轮子，Univer 提供了可组合的底层框架。

核心功能：一是多品类办公能力统一运行时，覆盖表格、文档、幻灯片、关系表、看板，PDF 也在规划中；二是基于 Canvas 的高性能渲染与插件化架构，可按需拼装功能；三是内置公式引擎，支持表格计算逻辑；四是一套 Facade API 同时适配浏览器和 Node.js，前后端同构；五是支持自定义插件、命令、服务和 UI 组件来扩展行为。

适用场景：适合把表格/文档编辑嵌入 SaaS 产品、内部工具、BI 工作流或 AI 应用的团队；也适合需要在服务端用同一套架构处理工作簿/文档的场景。尤其值得关注的是它把自己定位为"AI Agents 的 Office 底座"，人和 AI 可以在同一份文件上协作，适合做 Agent 生成仪表盘、交互式报表这类应用。

技术亮点：用 TypeScript 编写，Canvas 渲染带来了比 DOM 方案更可控的性能表现；插件化 + 命令/服务分层让功能解耦、可裁剪；浏览器与 Node.js 共享同一套存储和计算运行时，是它区别于普通前端表格库的关键；公式引擎和 Facade API 的统一抽象，降低了跨端和跨品类复用的成本。

上手建议：先读官方文档 (docs.univer.ai) 和 Showcase，从预设 (presets) 快速跑通一个最小示例，再按需引入插件。想深入可参考开源的 Univer Workspace 作为完整集成范例。贡献方面，建议从 issues 和 Discord 社区入手，熟悉其插件与命令体系后再提交 PR。

#5

C++

NEW

tensorflow

An Open Source Machine Learning Framework for Everyone

Stars **200,363** Forks **77,553** Today **+31**

<https://github.com/tensorflow/tensorflow>

项目简介：TensorFlow 是 Google Brain 团队开源的端到端机器学习平台，旨在为研究者和开发者提供从研究实验到生产部署的完整工具链。它解决了机器学习模型构建、训练、推理全流程中框架碎片化的问题，让不同规模的团队都能用统一生态完成 ML 应用开发。

核心功能：提供稳定且全面的 Python 与 C++ API，涵盖张量运算、自动微分、神经网络层等基础能力；内置 Keras 高阶接口，支持快速搭建和训练模型；支持 GPU/TPU 加速及设备插件（DirectX、macOS Metal）；配套 TensorBoard 可视化、TFX 生产流水线、Lite/JS 端侧部署等扩展生态；提供 tensorflow-cpu 轻量包和 nightly 测试包，灵活适配不同环境。

适用场景：学术研究者可用于神经网络前沿实验；工业开发者可将其部署到推荐系统、图像识别、自然语言处理等生产服务；边缘计算场景可借助 TensorFlow Lite 落地移动端与嵌入式设备；教学场景中也是入门深度学习的常用框架。

技术亮点：核心用 C++ 实现以保证性能，同时通过多语言绑定暴露给上层；采用数据流图与即时执行（Eager Execution）双模式，兼顾灵活调试与图优化部署；完善的 CI、模糊测试（OSS-Fuzz）、OpenSSF Scorecard 等安全与质量体系，体现大型开源项目的工程规范。

上手建议：使用者可直接 `pip install tensorflow` 并跟随官方教程跑通第一个示例；贡献者应先阅读 CONTRIBUTING.md 和 Code of Conduct，从 GitHub Issues 中挑选标签友好的问题入手，并通过 TensorFlow Forum 与社区沟通，避免直接提交大型改动。

#6

Python

NEW

ai-engineering-from-scratch

Learn it. Build it. Ship it for others.

Stars **57,987** Forks **10,061** Today **+828**

<https://github.com/rohitg00/ai-engineering-from-scratch>

项目简介：这是一个面向 AI 工程领域的开源课程项目，目标是让学习者从零开始系统掌握 AI 工程技能。它针对"84% 学生已在用 AI 工具，但只有 18% 觉得自己能专业使用"这一落差，提供从数学基础到 LLM 应用、Agent 开发的完整学习路径。

核心功能：提供 523 节课程、20 个阶段、约 342 小时的学习内容，覆盖 Python、TypeScript、Rust、Julia 四种语言；每节课都产生一个可复用工件（提示词、技能、Agent、MCP 服务器）；支持按目标选择学习路径（如"我想做 Agent"或"我想做 LLM 应用"）；提供 GitHub 与官网双入口，课程代码一致；包含多语言 landing page（中、日、韩、法、德等 12 种语言）。

适用场景：适合想系统入门 AI 工程的开发者、希望从传统软件工程转向 AI 方向的工程师、以及需要构建生产级 LLM 应用或 Agent 的团队。学习者可以按"基础→数学→LLM→Agent→MCP"的路径逐步深入，也可以直接跳到自己关心的阶段。

技术亮点：课程强调"手写实现"而非只调 API，每节课都要求产出可复用工件，强调工程落地而非纯理论；支持多语言课程页面（机器翻译在独立分支），课程内容与网站共享同一套代码；项目还关联了作者另一个高星项目 Agent Memory，形成生态协同。

上手建议：先访问官网或 README 中的"选择你想构建的目标"表格，根据自身基础选一条路径（如完全新手从 Phase 0 开始，有 Python 基础从 Phase 1 开始）。想贡献的话，可以从翻译分支或课程工件的改进入手，项目采用 MIT 协议，门槛较低。

#7

Go

NEW

openbao

OpenBao is a software solution to manage, store, and distribute sensitive data including secrets, certificates, and keys.

Stars **7,899** Forks **588** Today **+360**

<https://github.com/openbao/openbao>

项目简介：OpenBao 是一个用于管理、存储和分发敏感数据（如密钥、证书、数据库凭证等）的开源软件，本质上是一个社区主导的密钥管理平台。它解决的是现代分布式系统中“谁在访问什么密钥、如何安全轮转和审计”这一核心难题，让团队无需自行构建一套定制的密钥管理方案。

核心功能：一是**安全存储**，任意键值对密钥在写入持久化存储前会被加密，即使底层存储被攻破也无法直接读取明文；二是**动态密钥**，可按需为 AWS、SQL 数据库等系统生成临时凭证，并在租约到期后自动吊销；三是**加密即服务**，提供加解密能力而不落盘数据，让开发者无需自行设计加密方案；四是**租约与续期**，所有密钥都绑定租约，支持自动吊销和 API 续期；五是**级联吊销**，可按用户、类型等维度批量吊销密钥树，便于入侵时的快速封锁。

适用场景：适用于需要集中管理密钥和凭证的云原生团队、DevOps 工程师、安全团队以及采用微服务架构的组织。典型场景包括：为应用动态发放数据库短期凭证、集中管理 TLS 证书和 API Key、在合规审计要求下记录密钥访问日志，以及在不信任底层存储的环境中安全保存敏感配置。

技术亮点：项目采用 Go 语言编写，天然适合高并发和跨平台部署；支持多种存储后端（本地磁盘、PostgreSQL 等），架构上解耦了存储与加密逻辑。作为 Linux 基金会 OpenSSF 旗下的项目，它强调开放治理和 OSI 认可的开源许可证，并已获得 OpenSSF Scorecard 和 Best Practices 徽章，在供应链安全和社区治理方面有较高透明度。

上手建议：建议先阅读官网文档（openbao.org/docs）了解核心概念和部署方式，再通过本地 Docker 或二进制快速启动一个开发实例进行体验。如果想参与贡献，务必先阅读 CONTRIBUTING.md，并加入 Zulip 聊天室中的讨论频道，从文档改进或小型 issue 入手逐步熟悉代码库和社区流程。

#8

Rust

NEW

buzz

A hive mind communication platform

Stars **34,728** Forks **4,592** Today **+175**

<https://github.com/block/buzz>

项目简介：Buzz 是一个可自托管的协作工作空间，核心理念是让人类和 AI Agent 在同一个“房间”里平等协作。它试图解决的问题是：当前团队协作被割裂在聊天工具、代码托管平台、CI 面板、机器人脚本等七八个互不相通的服务中，而 Buzz 用一套统一的底层协议把这些能力整合到一起。

核心功能：一是人机同室协作，Agent 以独立身份（独立密钥、独立审计轨迹）加入频道，而非作为“机器人插件”存在；二是统一事件日志，所有消息、代码补丁、CI 结果、审批、Git 事件都是同一种签名事件，可被统一搜索和审计；三是 Agent 拥有与人类同等的操作面，能开仓库、发补丁、审查代码、运行工作流、创建频道甚至编排其他 Agent；四是“分支即房间”，把补丁、CI、审查和合并决策聚合在一个频道中，让代码存在的原因本身成为可追溯的记录。

适用场景：适合希望将 AI Agent 深度嵌入研发流程的工程团队，尤其是对数据主权有要求、倾向自托管方案的团队。典型场景包括：凌晨故障排查时让 Agent 检索历史事件并给出有据可查的答案、让 Agent 在受限身份下自动分类 Bug、以及将功能分支的全生命周期管理集中到一个可搜索的协作空间中。

技术亮点：项目基于 Nostr 中继协议构建，用 Rust 实现，这意味着每条消息、每次审批、每个 Git 事件都是同一格式的签名事件，天然具备统一的身份模型和审计能力。单中继部署对应单社区，多租户托管时后端可共享 Postgres/Redis/对象存储，但语义边界仍按社区隔离——这种“URL 即工作空间”的设计简洁而有力。

上手建议：建议先阅读仓库中的 VISION.md 和 ARCHITECTURE.md 理解设计哲学与系统架构，再通过 README 中的自托管部署指引搭建一个单中继实例进行体验。如果想贡献代码，Rust 项目结构清晰，可从事件处理或 Agent 权限模型相关的 issue 入手。

#9

TypeScript

NEW

vscode

Visual Studio Code

Stars **192,981** Forks **43,470** Today **+78**<https://github.com/microsoft/vscode>

项目简介：这是微软开源的 Visual Studio Code 源代码仓库（代号 Code - OSS），本质上是一款轻量级但功能强大的跨平台代码编辑器。它解决了开发者在编辑、构建、调试全流程中对高效工具的需求，同时通过开源模式让社区共同参与产品演进。

核心功能：全面的代码编辑与导航能力，支持语法高亮、智能补全、跳转定义等；内置轻量级调试功能，可直接在编辑器中断点调试；丰富的扩展模型，允许第三方通过插件扩展语言支持和工具集成；跨平台支持 Windows、macOS 和 Linux，且每月发布新版本持续迭代。

适用场景：几乎覆盖所有软件开发者的日常编码工作，尤其适合 Web 前端、Node.js、TypeScript 等现代技术栈的开发者。也适用于需要轻量编辑器而非重型 IDE 的场景，以及希望通过扩展定制个人开发环境的用户。

技术亮点：整个项目以 TypeScript 为主要语言构建，采用 Electron 框架实现跨平台桌面应用；架构上分离了核心编辑器与语言服务，通过 Language Server Protocol (LSP) 和 Debug Adapter Protocol (DAP) 实现语言支持的标准化接入，这是其扩展生态繁荣的关键设计。

上手建议：普通用户直接从官网下载安装即可使用；若想贡献代码，建议先阅读 wiki 中的 How to Contribute 文档，从构建源码和运行测试开始，再挑选标记为 help-wanted 或 good-first-issue 的 issue 入手；翻译贡献也是低门槛的参与方式。

#10

PowerShell

NEW

reverse-skill

Reverse Engineering / Authorized Penetration Testing / Security Research Skill Router
Pack AI-powered routing + On-demand toolchain bootstrapping + Self-evolving
knowledge base Supports Claude Code, Kiro, Cursor, Cline, and other AI coding
clients 逆向/渗透/安全技能路由包 - AI 自动路由 + 按需自举工具链 + 自动进化经验库 | 支持
Claude Code / Kiro / Cursor / Cline 等代码 AI 客户端

Stars **37,777** Forks **5,253** Today **+409**

<https://github.com/zhaoxuya520/reverse-skill>

项目简介：reverse-skill 是一个面向 AI 编程客户端的"逆向/渗透/安全技能路由包"，本质上是给 Claude Code、Cursor、Cline 等 AI Agent 装上一套安全领域的"作战手册"。它解决的核心问题是：AI 面对 APK、ELF 二进制、前端 JS 加密、PCAP、CTF 题目时往往不知道该用 jadx 还是 Frida、该走哪套流程，只能瞎猜命令；这个项目把方法论、工具链和经验库组织成可路由的技能模块，让 AI 按图索骥地执行。

核心功能：

- **AI 自动路由：**通过 MASTER-ROUTING / routing.md 等结构化规则（44 条 R0-R45 规则），根据任务类型把 AI 导向对应的场景技能模块。
- **按需自举工具链：**检测本机可用的工具（jadx、apktool、Frida、IDA、BurpSuite 等）、MCP 服务器和脚本，缺什么补什么，而不是假设环境已就绪。
- **自进化经验库：**把每次任务的 timeline、Evidence→Finding→Path 沉淀成 field-journal，避免重复踩坑。
- **标准化作业契约：**case-init / scope.md 强制先确认授权与网络配置，未就绪前不允许对目标执行动作，内置合规边界。
- **客户端中立：**核心路由与具体 AI 客户端解耦，可适配多种代码 AI 工具。

适用场景：主要面向做授权渗透测试、CTF 竞赛、恶意样本分析、APK/二进制逆向、前端加密破解的安全研究人员和红队工程师。典型用法是在 Claude Code 或 Cursor 里直接下达"分析这个 APK"或"看看这段 JS 加密"，由路由包接管后续的方法选择和工具调用。也适合需要把安全作业流程标准化、可复现的团队。

技术亮点：用 PowerShell 作为主要实现语言，配合 Markdown 驱动的规则配置，把"路由逻辑"和"客户端适配"分层解耦，这是它能跨平台（Windows + Ubuntu CI）且客户端中立的关键。175 个回归测试用例 + 跨平台 CI 说明它在把"提示词工程"往"可测试的软件工程"方向做，这在同类 AI 技能包里比较少见。强调"授权 + scope 先行"的设计也体现了对合规的重视。

上手建议：先读 README_AI.md（作者明确要求 AI Agent 从这里入手），再走一遍 skills/MASTER-ROUTING.md 的主路由梯和 skills/routing.md 的完整矩阵，理解规则体系。人类用户建议从官网在线教程和 Getting Started 入手，用一个简单的 CTF 或自己的测试目标跑通全流程。想贡献的话，可以从补充场景技能模块、增加工具适配或提交回归测试用例切入，注意遵守 MIT 协议和项目的授权使用边界。

数据来源: github.com/trending

报告日期: 2026-09-26

由 DeepSeek AI 提供智能分析 | 自动生成